Version 0.2.5

Presentate

Simple dynamic presentation in **Typst**,
with flexible package integration framework.

User Manual

GitHub: https://github.com/pacaunt/typst-presentate
Compiled on 04 March 2026

Contents

1 Introduction 3
  1.1 Acknowledgements 3
2 Getting Started 4
  2.1 Creating slides 4
  2.2 Simple animation 4
  2.3 Control the show and hide: index specification 5
  2.4 Time traveling with relative indices 6
  2.5 Specific functions for `list` and `enum` 7
3 Advanced Usage 9
  3.1 Groupping pauses: the `fragments` function 9
  3.2 Change the hiding method: the `hider` argument 9
  3.3 Change the appearance of the content: the `transform` function 10
  3.4 Precise control of the animation: `motion` and `tag` function 11
    The `motion` workspace 11
    Control Rules 12
4 Package Intregration Framework 12
  4.1 The `render` workspace 12
  4.2 `animation` module 12
5 Exposed Utilities 12
6 Themes 12

1 Introduction

`Presentate` is a Typst package for creating *dynamic* PDF presentation that is compatible with other packages. The word *dynamic* means the compiled PDF contains *animated* content. But PDF is a static document format, how can it contains animations? Presentate will look into the content, and create a set of pages that reveal or hide some content based on the current number of frames called *subslides*, so that when going through the pages, it seems like the content is showing or hiding like a simple animation.

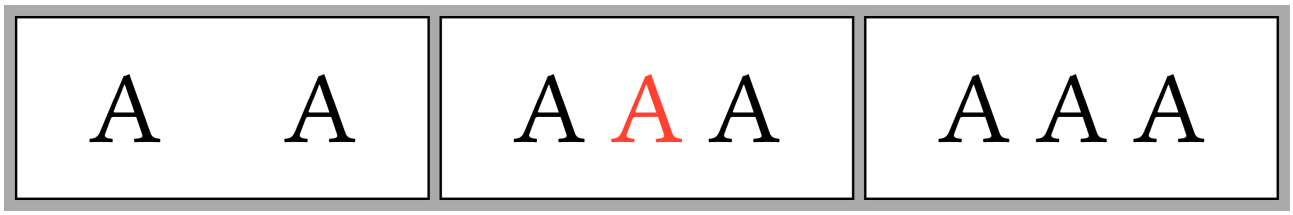
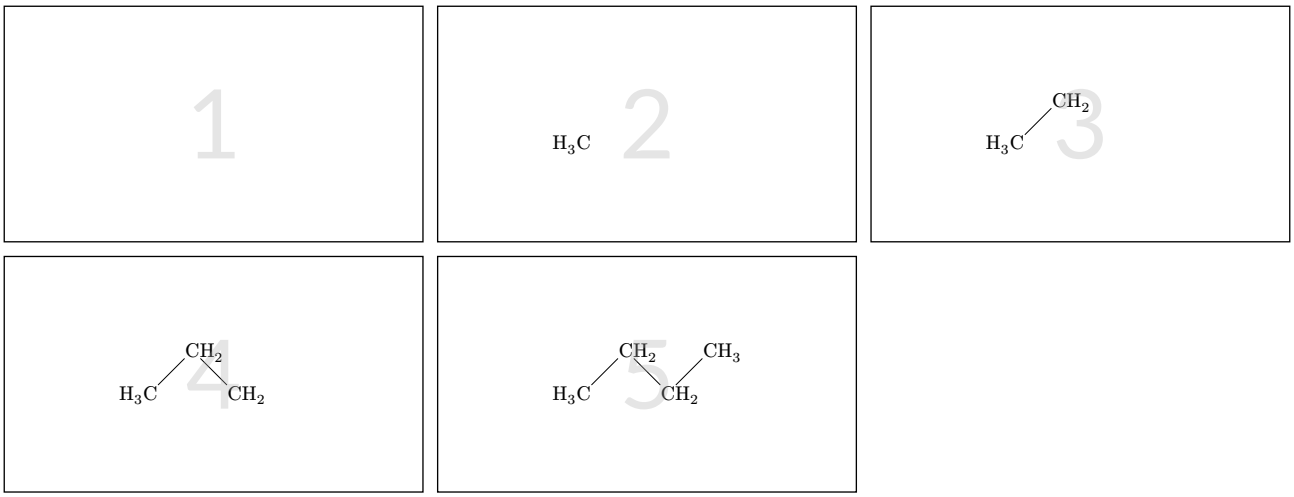


Figure 1: Example of fake animation in PDF format. Each page contains its own content and rules controlling them to hide, show, or decorated in some ways.

Dynamic PDF presentation packages have been in the Typst community for a long time, with the similar strategy for creating the animations, so why Presentate existed? The answer lies on the fact that presentation contains many kinds of visualization like plots, drawings, or chemistry molecules. These elements are usually created from other packages which have their own functions and data types. Therefore, to create animation with these visualization, the presentation framework must be able to handle *any* data types:

- be able to *hide* or *show* any elements, not only `content`,
- be able to generate the number of required subslides *automatically*.

Presentate is created for that purpose. It provides not only the slide animation engine, but also a framework and functions to interact with content from other packages. For example, here is some animation of drawing molecules integrated with the Alchemist[1] package.



Note that the gray text represents the subslide (frame) of that page.

1.1 Acknowledgements

The package was created by mixing my original motivation and insprations from many existing presentation packages. Thanks to:

- Polylux[2] for `subslide` implementation and pdfpc support,

[1] https://typst.app/universe/package/alchemist

- Touying[3] for idea of render frame, fake frozen states, and
- Minideck[4] for `only`, and `uncover` functions.
- Navigator[5] for the navigation system, progressive-outline, and the structured themes.

2 Getting Started

Import the package by

```
#import "@preview/presentate:0.2.5": *
```

Or download a zip file of Presentate's repository[6], install the `src` folder and import the package by

```
#import "src/export.typ": *
```

Then, setting your own styles, for example,

```
#set page(paper: "presentation-16-9") // you can change to 4-3
#set text(size: 25pt)
// your presentation begins here!
```

2.1 Creating slides

A slide can be created by using the `slide` function. For example,

```
#slide[
  Hello, this is presentate.
]
```

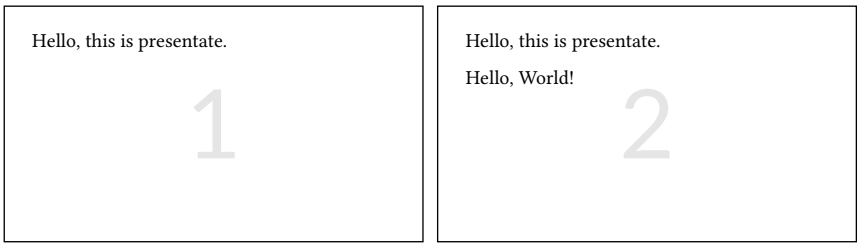
2.2 Simple animation

In the slide function, you can use the animation functions to control the behavior of the content on the slide. The simplest animation is to show some content after another content, this can be done by `pause` function:

```
#slide[
  Hello, this is presentate. #show: pause

  Hello, World!
]
```

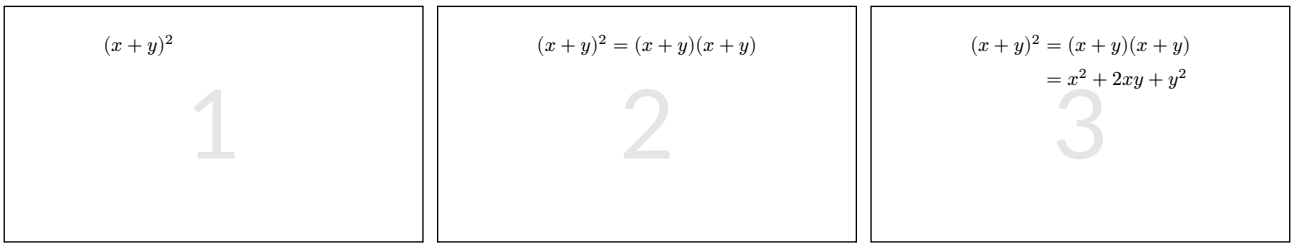
yields



[2] https://github.com/polylux-typ/polylux
[3] https://github.com/touying-typ/touying
[4] https://github.com/knuesel/typst-minideck
[5] https://github.com/eusebe/typst-navigator
[6] https://github.com/pacaunt/typst-presentate

Each content wrapped by the `pause` function will be revealed one by one on each frame. This function can be used in `math.equation` as well:

```
#slide[
  $
    (x + y)^2 pause(&= (x + y)(x + y)) \
              pause(&= x^2 + 2x y + y^2)
  $
]
```

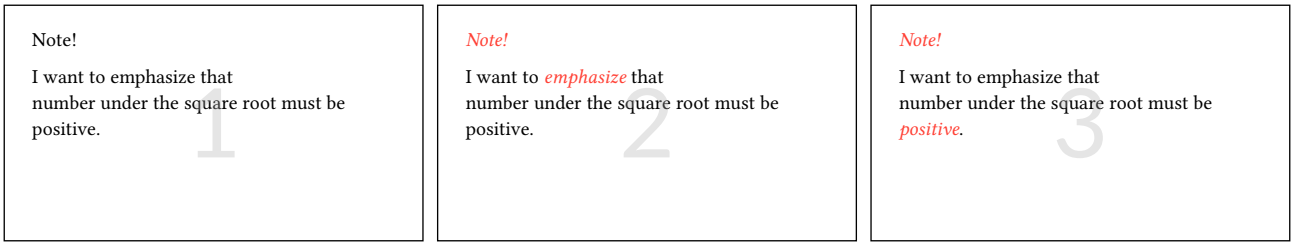


2.3 Control the show and hide: index specification

If, for example, you want to emphasis a text on a slide, you can use the `alert` function. This function, by default, will *emphasize* the content by wrapping it in the typst's `emph` function and become *alerted* on the specified subslide.

```
#show emph: set text(fill: red) // for apparent looking
#slide[
  #alert(from: 2)[Note!] // stays alerted from subslide 2

  I want to #alert(2)[emphasize] that \
  number under the square root must be #alert(3)[positive].
]
```



In the example, `alert` function accepted an integer and it made the text red only when the subslide number is the same as the specified number. This is called **index specification** where the index (pl. indices) is the subslide number or *frame* number. Indices can be specified in many ways:

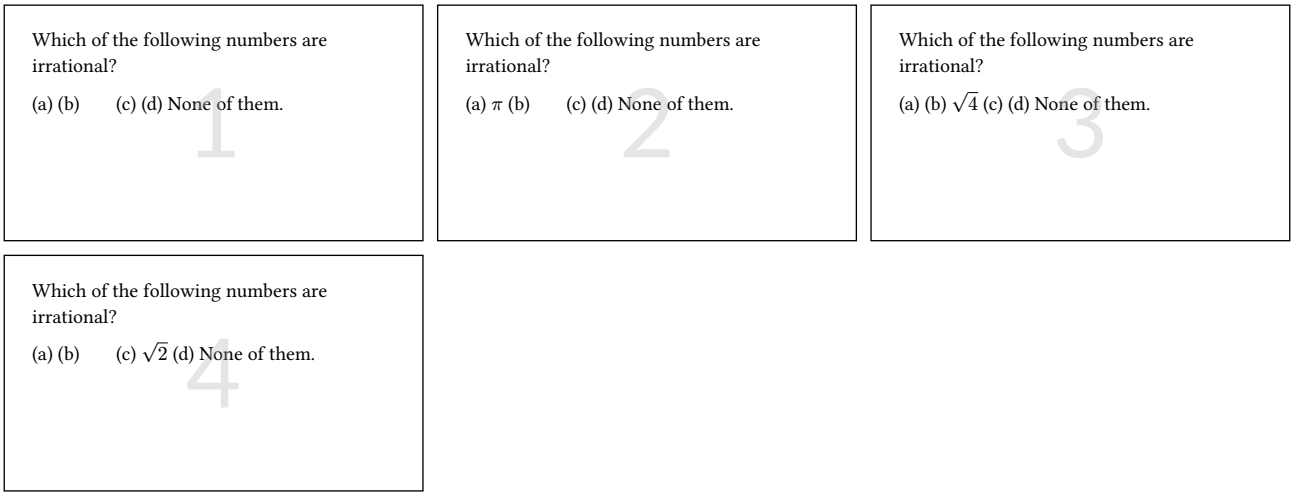
- It can be an integer, to alert the content only at that subslide.
- It can be `from: int`, to start alerting since the `int`th subslide.
- It can be multiple integers like `alert(1, 4, 5, body)` to only alert on subslide 1, 4, and 5.
- It can be *both* integers and `from: int`, to show the modified content on the specified subslide and after the `int` subslide specified by the `from` argument.

Apart from `alert`, if you have ever used `beamer` in LaTeX before, you probably know the `uncover` and `only` functions. The `uncover` only *uncover* the content on the spcified subslides, and hide the content otherwise, with *space preserved*, while `only` function also works with the same logic, but its hiding method is to completely remove the content out, so no space is preserved.

```
#slide[
```

```
  Which of the following numbers are irrational?

  (a) #only(2, $pi$)
  (b) #uncover(3, $sqrt(4)$)  // The space here is peserved.
  (c) #only(4, $sqrt(2)$)
  (d) None of them.
]
```



2.4 Time traveling with relative indices

Most of the time when creating presentation, the step of revealing the content is either

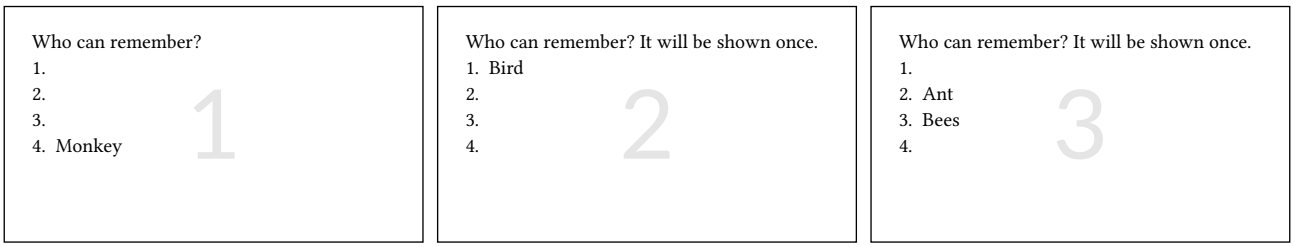
- showing at the same subslide as previous animation, or
- showing at the next subslide of the recent animation.

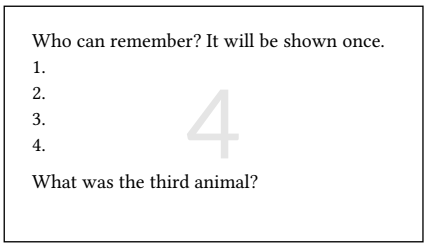
Presentate have a special type of index called **relative indices**, which are indices relative to the current number of pauses appear on the slide. There are 3 forms of relative indices:

1. `none` means the animation will happen at the same subslide as the current pause.
2. `auto` means the animation will happen at the *next* subslide after the current pause.
3. `(rel: int)` means the animation will happen at the `int` subslide away from the current pause.

For example,

```
#slide[
  Who can remember?
  #pause[It will be shown once.] // first pause
  + #uncover(none)[Bird] // with the pause
  + #uncover(auto)[Ant] // after the pause
  + #uncover((rel: 1))[Bees] // also after the pause
  + #uncover((rel: -1))[Monkey] // before the first pause
  #uncover(from: (rel: 2))[What was the third animal?]
]
```



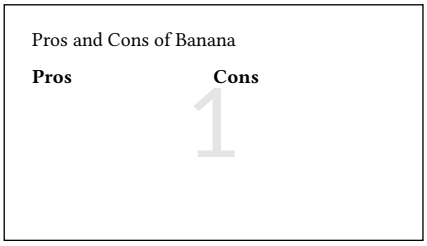


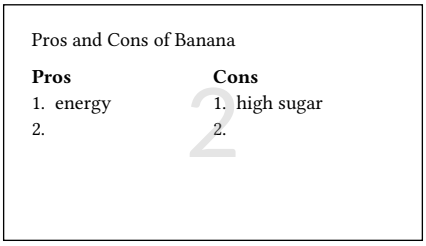
As you can see, the last text that was shown on subslide 4 was a little bit inconvenient to write `(rel: 2)` to make it shown when every animation was finished. It would be nice if we can use `pause` at the end and make it 'see' the previous `uncover`s. You can make the next pause reveal a content after (almost) any animation functions by specifying `update-pause: true` into its argument.

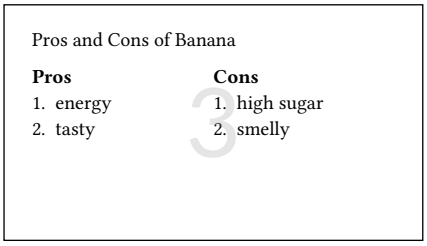
```
#slide[
    Who can remember?
  #pause[It will be shown once.] // first pause
  + #uncover(none)[Bird]
  + #uncover(auto)[Ant]
  + #uncover((rel: 1), update-pause: true)[Bees] // update the last animation
  + #uncover((rel: -1))[Monkey]
  #pause[What was the third animal?] // so this shows later
]
```

With this interaction between the current number of pauses and the `update-pause` argument, you can set the pauses to *any* number to modify the animation. For example, making the content shown in sync side by side:

```
#slide[
  Pros and Cons of Banana
  #grid(columns: (1fr,) * 2)[
    *Pros* #show: pause
    + energy
    #show: pause
    + tasty
  ][ // [] is the dummy content.
    #uncover(1, [], update-pause: true)
    *Cons* #show: pause
    + high sugar
    #show: pause
    + smelly
  ]
]
```





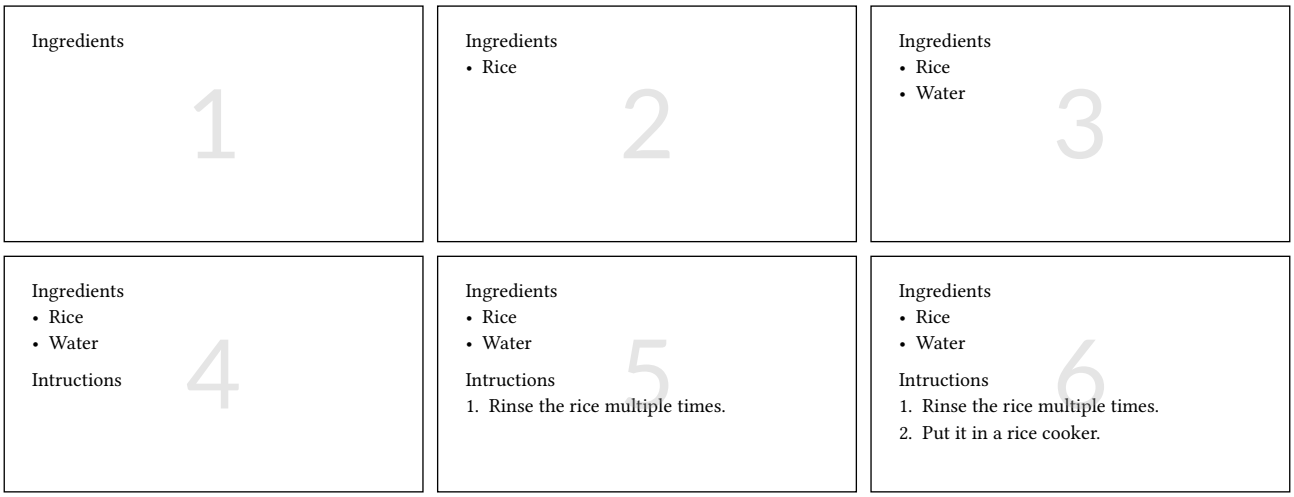


2.5 Specific functions for `list` and `enum`

In the last example, you can see that the number in `enum` did not hide properly. Since Typst treats the marker of `list` and number label of `enum` in different context from its body, Presentate implements

some (hacky) way to properly hide them without wiggling. To show the list/enum item by item, Presentate provides a function called `step-item`:

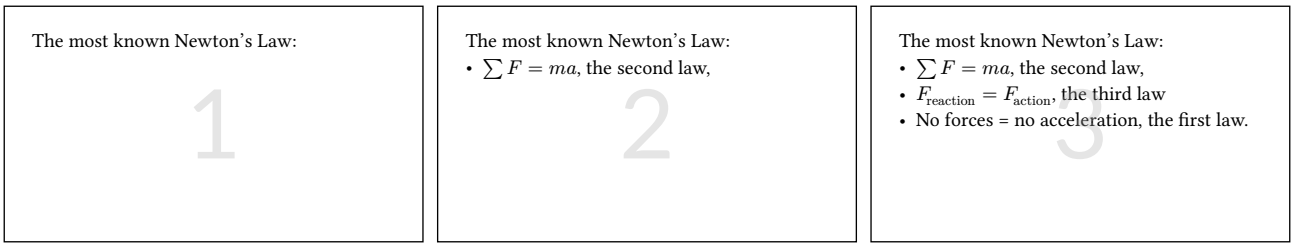
```
#slide[
  Ingredients
  #step-item[
    - Rice
    - Water
  ]
  #show: pause
  Intructions
  #step-item[
    + Rinse the rice multiple times.
    + Put it in a rice cooker.
  ]
]
```



Each of the item's body and its marker are wrapped in `pause` function, and synchronize the animation by using the `update-pause` hack. Therefore, you can nested them indefinitely, but it always update the pauses.

If you want to reveal the items chunk by chunk, Presentate also have the `reveal-item` function. This function can show item in group by group without breaking them. However, since the implementation of `reveal-item` requires additional Typst's state, it can decrease the speed of Typst compilation. So use this function sparingly.

```
#slide[
  The most known Newton's Law:
  #reveal-item[
    - $sum F = m a$, the second law,
  ][
    - $F_"reaction" = F_"action"$, the third law
    - No forces = no acceleration, the first law.
  ]
]
```



3 Advanced Usage

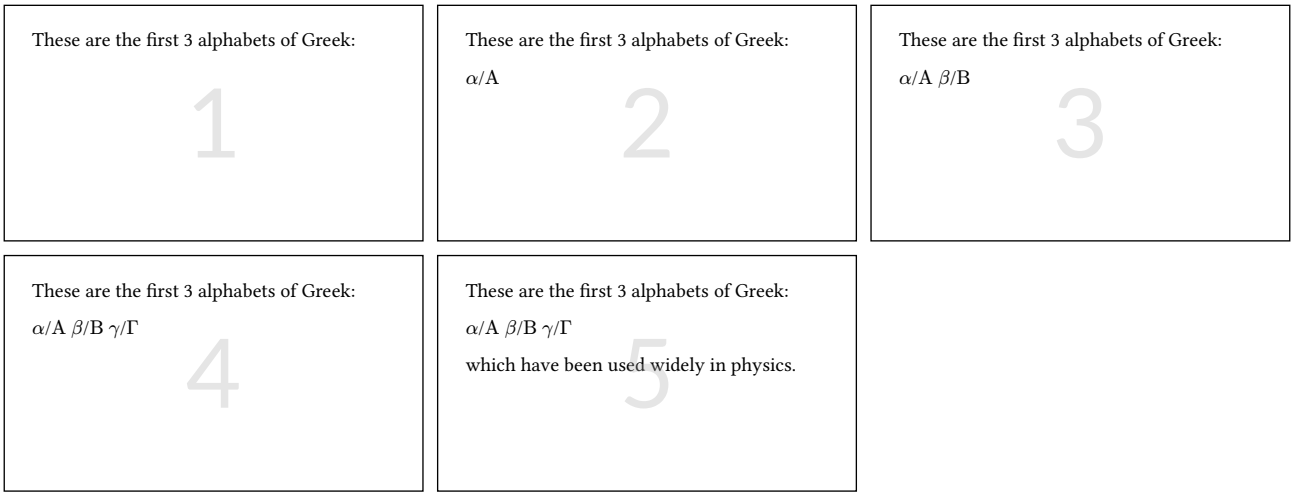
3.1 Groupping pauses: the `fragments` function

This function works like a shorthand for multiple pause/uncover calls. Moreover, you can choose whether to make it update the current pause by `update-pause` argument.

```
#slide[
  These are the first 3 alphabets of Greek:

  #fragments[$alpha$/$Alpha$ ][$beta$/$Beta$ ][$gamma$/$Gamma$]
  #show: pause

  which have been used widely in physics. // the fragments update the pause.
]
```



3.2 Change the hiding method: the `hider` argument

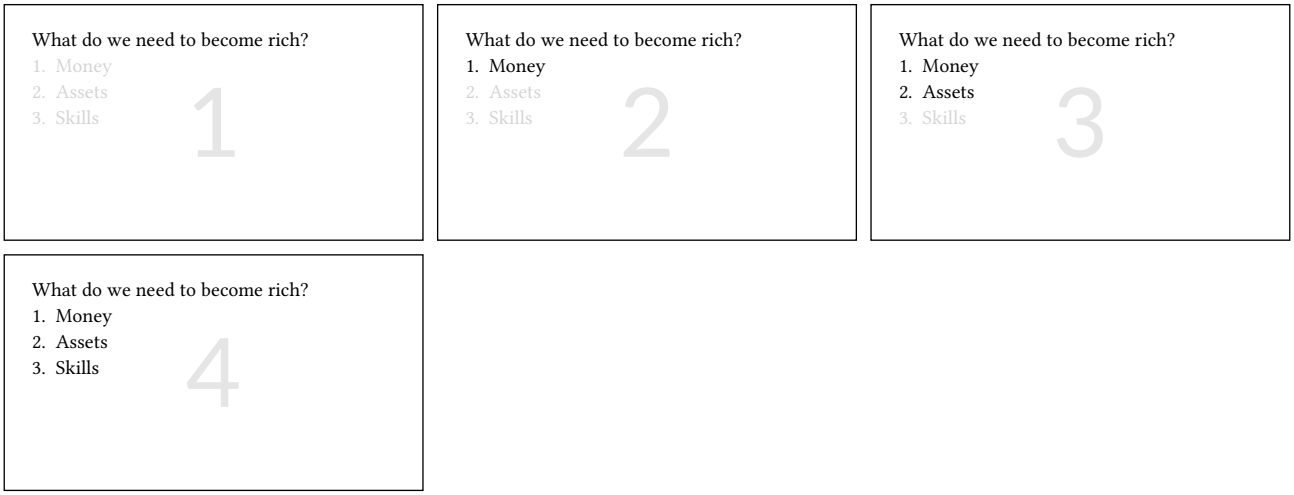
Every animation function in Presentation has a named argument `hider`, which is a function that used internally to *hide* the content. For example,

- `only` uses `it` ⇒ `none` as hider, which make the content disappear when it hides,
- `uncover`, `pause`, `step-item`, `fragments` uses native Typst's `hide` function, which preserves space.

If you want to change the hiding method of the text from completely hiding it to making it gray, you can use `text.with(fill: gray.transparentize(50%))` as the hider, like the following example,

```
#slide[
  What do we need to become rich?
  #step-item(hider: text.with(gray.transparentize(50%)))[
    + Money
    + Assets
    + Skills
```

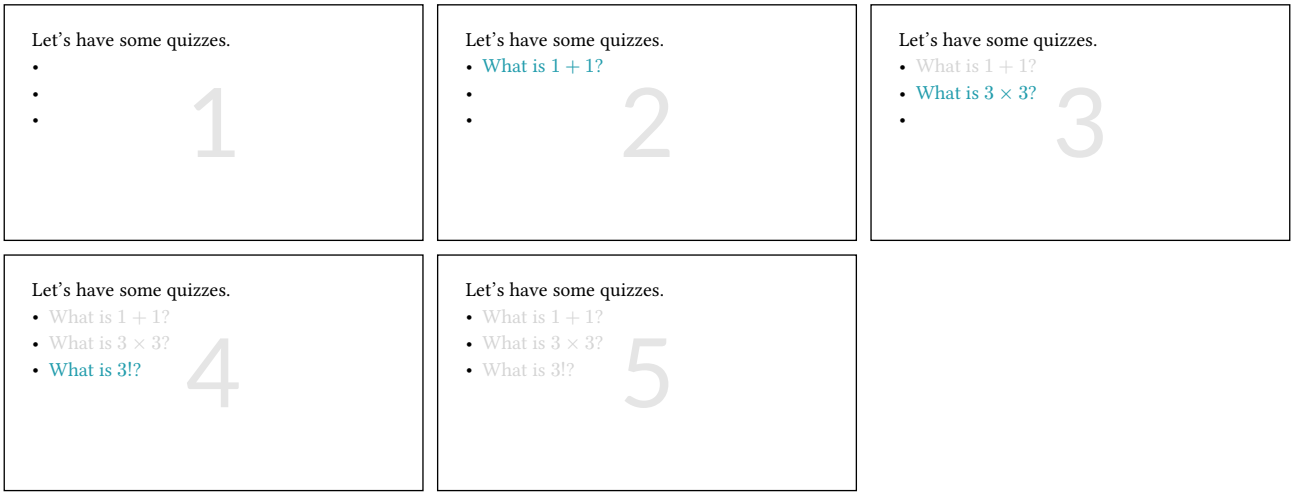
```
  ]
]
```



3.3 Change the appearance of the content: the `transform` function

`transform` requires a content as a base, it will be hidden by the `hider` until the specified index is reached (default is `auto`). Then, on each consecutive subslide, the content will transform by each function one by one. The following example shows how to emphasize content in a list by using `transform`.

```
#slide[
  #let yes = text.with(fill: eastern)
  #let no = text.with(fill: gray.transparentize(50%))
  Let's have some quizzes.
  - #transform([What is $1 + 1$?], yes, no)
  - #transform([What is $3 times 3$?], yes, no, start: none)
  - #transform([What is $3!$?], yes, no, start: none)
]
```



`start` argument tells when to start the animaton. Note that in this example, the grayed text was from `no` function, not the hider. The hider was still `hide` function, as you can see in the subslide 1.

3.4 Precise control of the animation: `motion` and `tag` function

The `motion` workspace

`motion` function provides a *workspace* for rendering content groups that are named by the `tag` function. The anatomy of this function is

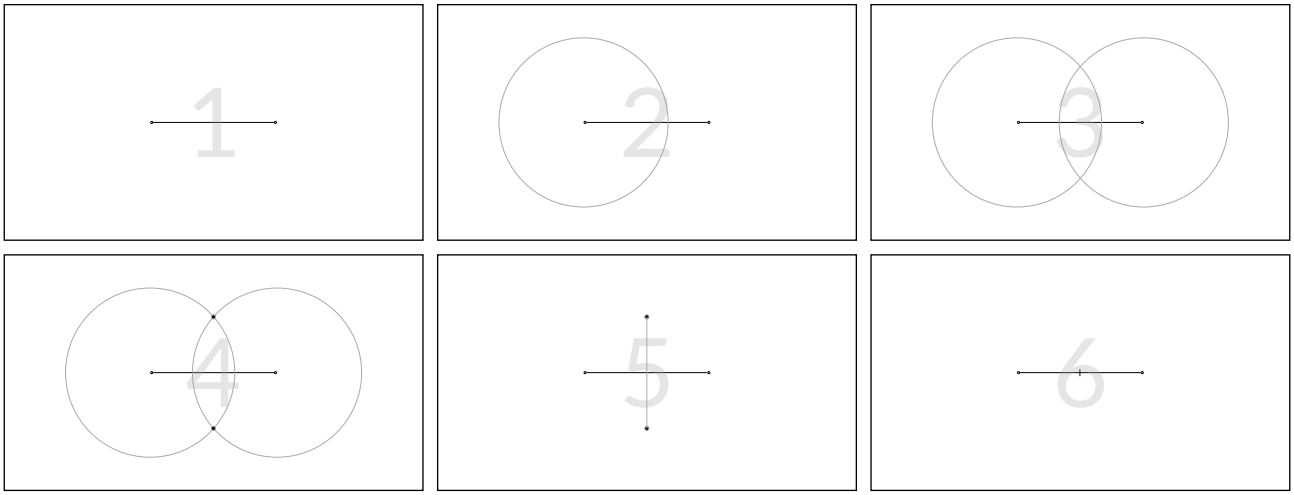
```
#motion(s => [
  // some content and `tag(s, name, body)`
], controls: (
  // rules to show the `tag`
))
```

Motion function accepts only one positional argument which is a **function** that receives the presentate's state `s` and return a content block. `motion` function have a `controls` argument to specify which content group should be shown or hide precisely subslide by subslide. The motion reads the `controls` rules, process them and send the showing information through the state `s`. This state `s` is read by the `tag` function to show or hide the content accordingly.

```
#import "@preview/cetz:0.4.2": canvas, draw

#slide[
  #set align(center + horizon)
  #motion(
    s => [
      #canvas({
        import draw: *
        scale(1.5)
        stroke((thickness: 2pt))
        line(name: "L", (0, 0), (6, 0), mark: (symbol: "o"))
        stroke((paint: gray))
        intersections("I", {
          tag(s, "circ1", circle("L.start", radius: 4))
          tag(s, "circ2", circle("L.end", radius: 4))
        })
        tag(s, "pt", {
          circle(radius: .1, "I.1", fill: black)
          circle(radius: .1, "I.0", fill: black)
        })
        tag(s, "cut", line("I.0", "I.1"))
        tag(s, "mark", {
          stroke(black)
          mark("L.mid", "L.end", symbol: "|", width: .5)
        })
      })
    ],
    hider: draw.hide.with(bounds: true),
    controls: (
      (), // don't show any group
      "circ1.start", // show the `circ1`
      "circ2.start", // show the `circ2`
      "pt.start", // show the `pt`
      ("circ1.stop", "circ2.stop", "cut"),
      // stop showing `circ1` and `circ2`, and show the `cut` once
      ("mark.start", "pt.stop"), // show the `mark` and stop showing the `pt`
    ),
  )
```

```
]
```



The example shown earlier was created with integration of CeTZ[7] pacakge, showing the capability to integrate with other packages. This integration works because of the `hider` argument in the motion function is the CeTZ's hide function, which is compatible for its own elements. Note that for more complex illustrations, you can provide custom `hider` to the `tag` function to use a specific hider for that group.

Control Rules

4 Package Intregration Framework

4.1 The `render` workspace

4.2 `animation` module

5 Exposed Utilities

6 Themes

[7] https://typst.app/universe/package/cetz